

AEI – Final Project Report

Nandhini Ramachandran – C19299747

Project 1: Sensing Signal Processing

1. Signal Processing for One Ultrasonic Sensor

1.1 Problem Statement

This project focuses on signal processing for a single ultrasonic sensor by converting raw echo time measurements into an accurate and stable distance estimate in millimeters.

As per the project requirements, the system must:

- Sample the sensor echo time data as fast as possible.
- Filter noise in the time measurements using an appropriate filter (a Kalman filter is mandatory).
- Calibrate a fitting function that maps time (μs) to distance (mm) and determine its parameters.
- During testing, print distance (mm) and variance in real time, once the estimate stabilizes (small variance or minimal change), flash an LED, stop printing and output the total time cost.
- Operate over the test range $20\text{ cm} < d < 1.5\text{ m}$.

1.2 Technical Approach

The solution uses a three-stage pipeline: fast sampling of ultrasonic echo time, 1-D Kalman filtering on the time signal and calibration to convert filtered time to distance.

Sampling model: Each cycle triggers the ultrasonic sensor and measures the echo pulse width using Arduino's `pulseIn()`, producing a raw measurement in microseconds.

Filtering model (1-D Kalman filter on time):

- State x_k : the true (noise-free) echo time (μs).
- Process model: $x_k = x_{k-1} + w_k$ with process noise variance Q .
- Measurement model: $z_k = x_k + v_k$ with measurement noise variance R .
- Prediction: $\hat{x}^- = \hat{x}$, $P^- = P + Q$.
- Update: $K = P^- / (P^- + R)$, $\hat{x} = \hat{x}^- + K(z - \hat{x}^-)$, $P = (1-K)P^-$.

Calibration model (linear fit): $\text{distance_mm} = a \cdot \text{time_us} + b$.

Stop criterion: the filter is considered done when both posterior variance P is below a threshold and the estimate change between iterations is small for several consecutive samples

1.3 Hardware and Software Implementation

Hardware setup:

- Ultrasonic sensor: trigger on digital pin 13, echo on digital pin 12.
- Indicator LED on digital pin 9.
- Serial output at 9600 baud to log distance and variance.

Software Setup:

Data collection for calibration:

- Implemented a code in Arduino to collect data.
- Collected 100 raw echo-time measurements (μs) at a fixed known distance.
- It prints the most frequent value out of 100 samples to reduce the effect of outliers before storing the time_us for that distance.
- This is repeated for 28 different distances for each sensor separately.

```
// --- Collect 100 samples ---
for (int i = 0; i < NUM_SAMPLES; i++) {
    samples[i] = readUltrasonicTime();
    delay(10);
}

// --- Find most frequent value ---
unsigned long modeValue = 0;
int modeCount = 0;

for (int i = 0; i < NUM_SAMPLES; i++) {
    int count = 1;

    for (int j = i + 1; j < NUM_SAMPLES; j++) {
        if (samples[i] == samples[j]) {
            count++;
        }
    }

    if (count > modeCount) {
        modeCount = count;
        modeValue = samples[i];
    }
}
```

Real-time estimation:

- The coefficients a and b are estimated in MATLAB from the collected sensor data and exported as an Arduino header as calibration.h

```
% Your Data file (one row per measurement)
xlsxFile = 'Sensor2.xlsx';

% Column names in the Excel
timeColName = 'time_us'; % echo time in microseconds
distColName = 'true_distance_mm'; % ground-truth distance in mm

% Output header file name
hdrFile = 'calibration2.h';

%% ===== LOAD DATA =====

T = readtable(xlsxFile);

% Check that required columns exist
if ~all(ismember({timeColName, distColName}, T.Properties.VariableNames))
    error('Excel must contain columns "%s" and "%s".', timeColName, distColName);
end

% Directly use the raw samples
time_us = T.(timeColName);
dist_mm = T.(distColName);

%% ===== LINEAR REGRESSION =====

p = polyfit(time_us, dist_mm, 1);
a = p(1);
b = p(2);

fprintf('Calibration result:\n');
fprintf('distance_mm = %.10f * time_us + %.10f\n', a, b);

%% ===== WRITE calibration.h =====

fid = fopen(hdrFile, 'w');
if fid == -1
    error('Cannot open %s for writing.', hdrFile);
end
fprintf(fid, '// Auto-generated calibration header\n');
fprintf(fid, '// Generated by MATLAB from %s\n\n', xlsxFile);
fprintf(fid, '#ifndef CALIBRATION2_H\n');
fprintf(fid, '#define CALIBRATION2_H\n\n');
fprintf(fid, '// Linear calibration model:\n');
fprintf(fid, '// distance_mm = CAL2_A * time_us + CAL2_B\n\n');
fprintf(fid, 'const float CAL2_A = %.10ff;\n', a);
fprintf(fid, 'const float CAL2_B = %.10ff;\n\n', b);
fprintf(fid, '#endif // CALIBRATION2_H\n');

fclose(fid);
fprintf('\nWrote header file: %s\n', hdrFile);
```

calibration.h file:

```
// Auto-generated calibration header
// Generated by MATLAB from Sensor2.xlsx

#ifndef CALIBRATION2_H
#define CALIBRATION2_H

// Linear calibration model:
// distance_mm = CAL2_A * time_us + CAL2_B

const float CAL2_A = 0.1749264464f;
const float CAL2_B = 10.7584005379f;

#endif // CALIBRATION2_H
```

- `readTimeUs()` triggers the sensor (10 μ s pulse) and measures echo time using `pulseIn(echoPin, HIGH, 30000)`. `VAR_THRESHOLD_TIME` = 25.0, `DELTA_TIME_THRESHOLD_US` = 10.0, and `STABLE_COUNT_REQUIRED` = 10, `Q_PROCESS` = 0.0, `R_MEAS` = 2.0, and `P_INIT` = 800.0
- Applied the 1-D Kalman filter update each loop iteration to obtain a filtered time estimate $\hat{x}$ and variance P .
- Converted the filtered time to distance using the calibration constants: $\text{distance_mm} = \text{CAL_A} * \hat{x} + \text{CAL_B}$ (from the `calibration.h` header file).
- Printed: "distance_mm, variance" each iteration.
- When stable, flashes the LED three times, stops printing, and outputs total elapsed time (`Time2 - Time1`) in milliseconds.

1.4 Experimental Results

Filtering behavior: During live operation, the printed variance P decreases as measurements accumulate. Once P is below the threshold (25.0) and the estimate change remains under 10 μ s for 10 consecutive iterations, the system indicates completion by flashing the LED and reports the total time cost in milliseconds. The exact convergence time depends on the sensor noise level, the chosen R/Q values, and the target distance within the required 20 cm-1.5 m range.

Sample Serial output:

```
512.34, 0.99
```

```
512.37, 0.67
```

```
512.40, 0.20
```

```
842 Milliseconds
```

2. Sensor Fusion of Multiple Ultrasonic Sensors

2.1 Problem Statement

This part extends Project 1.1 to multiple ultrasonic sensors and estimates a single true distance by fusing readings from at least two sensors.

The system must:

- Sample echo time data from 2 ultrasonic sensors as fast as possible.
- Calibrate each sensor.
- Fuse multiple distance measurements using a Kalman filter to estimate the true distance.
- Print fused distance (mm) and variance in real time, when stable (variance small and/or estimate change small), flash an LED, stop printing, and output total time cost.
- Operate over the test range $20\text{ cm} < d < 1.5\text{ m}$.

2.2 Technical Approach

Pipeline: acquire echo time from each ultrasonic sensor, convert each time measurement to distance using its calibration parameters and fuse the multiple distance measurements with a 1-D Kalman filter.

Per-sensor calibration (linear): for sensor i , $\text{distance_mm} = A_i \cdot \text{time_us} + B_i$. The coefficients are stored in header files and used on the Arduino for real-time conversion. Calculate an average.

Fusion model (1-D Kalman filter on distance):

- State y_k : true distance (mm).
- Process model: $y_k = y_{k-1} + w_k$, with process noise variance Q ($Q = 0.0$ for a static target).
- Measurements: $z1_k$ and $z2_k$ are the calibrated distances from sensor 2 and sensor 5 with measurement variances $R1$ and $R2$.
- Prediction: $y_{\text{pred}} = y_{\text{hat}}$, $P_{\text{pred}} = P + Q$.
- Sequential measurement updates (one update per sensor):
 - $K1 = P_{\text{pred}} / (P_{\text{pred}} + R1)$, $y1 = y_{\text{pred}} + K1(z1 - y_{\text{pred}})$, $P1 = (1 - K1)P_{\text{pred}}$
 - $K2 = P1 / (P1 + R2)$, $y_{\text{hat}} = y1 + K2(z2 - y1)$, $P = (1 - K2)P1$

2.3 Hardware and Software Implementation

Hardware setup:

- Ultrasonic sensor 2: trig pin 13, echo pin 12.
- Ultrasonic sensor 5: trig pin 11, echo pin 10.
- Indicator LED: pin 9.
- Serial logging: 9600 baud.

Software implementation details:

- `readTimeUs()` triggers the sensor with a 10 μ s pulse and measures echo pulse width using `pulseIn(30000)`. If no echo is received before timeout, returns NaN. `VAR_THRESHOLD_TIME` = 25.0, `DELTA_TIME_THRESHOLD_US` = 10.0, and `STABLE_N` = 10, `Q_PROCESS` = 0.0, `R_MEAS` = 2.0, and `P_INIT` = 800.0
- `setup()`: prints START, records `T0 = millis()`, reads both sensors once and initializes the fused estimate `y_hat` as the average of the two calibrated distances.
- `loop()`: reads both sensors sequentially (with a short delay between them), converts each time measurement to distance (`z1`, `z2`), runs the Kalman prediction + two sequential corrections and prints `y_hat` and `P` each iteration.
- When stable for `STABLE_N` iterations, sets `done=true`, blinks the LED 5 times, prints the total time cost (`T1-T0`) in milliseconds, then stops updating.

In `setup()`

```
// Initial Setup
if (!isnan(t4) && !isnan(t5)) {
    float z4 = CAL2_A * t4 + CAL2_B;
    float z5 = CAL5_A * t5 + CAL5_B;
    y_hat = 0.5f * (z4 + z5); // Average of the two sensors
} else {
    y_hat = 0.0f; // Fallback
}

P = P0;
prev_y = y_hat;
```

In `loop()`

```
// Calibration
float z1 = CAL2_A * t4 + CAL2_B;
float z2 = CAL5_A * t5 + CAL5_B;

// ----- Prediction -----
float y_pred = y_hat;
float P_pred = P + Q;

// ----- Correction 1 (Sensor 2) -----
float K1 = P_pred / (P_pred + R1);
float y1 = y_pred + K1 * (z1 - y_pred);
float P1 = (1.0f - K1) * P_pred;

// ----- Correction 2 (Sensor 5) -----
float K2 = P1 / (P1 + R2);
y_hat = y1 + K2 * (z2 - y1);
P = (1.0f - K2) * P1;

// Output
Serial.print(y_hat, 2);
Serial.print(", ");
Serial.println(P, 2);
```

2.4 Experimental Results

Fusion behavior: combining two independently calibrated sensors produces a smoother distance estimate than either raw sensor alone. The reported variance P decreases rapidly from the initial value ($P_0 = 800$) as measurements are incorporated.

With $Q = 0$ and small R values, the filter strongly trusts the measurements, so P typically drops to near the measurement-noise scale within a few iterations and then remains small while the target is stationary.

Convergence and time cost: once $P < 25$ and the fused distance change is < 10 mm for 10 consecutive iterations, the system flashes the LED and prints the elapsed time in milliseconds. The exact time depends on sensor noise, target reflectivity/angle, and the delays used between sensor reads.

Sample Serial output:

```
900.36, 0.99
```

```
900.10, 0.67
```

```
900.09, 0.20
```

```
733 Milliseconds
```

3. Conclusions and Discussions

3.1 Conclusions

Project 1.1: single sensor + 1D Kalman on time + linear calibration: produced a stable distance estimate by smoothing noisy echo-time readings with a Kalman filter and converting the filtered time to distance using a fitted linear model, the variance P drops quickly and the system can reliably detect when the estimate is stable, then flash the LED and report time cost.

Project 1.2: two sensors + per-sensor calibration + Kalman fusion on distance: improved robustness by combining two independently calibrated distance measurements into a single fused distance, fusion reduces the effect of individual sensor noise/outliers and typically yields a smoother estimate than either sensor alone, while still converging fast.

3.2 Discussions

Noise handling:

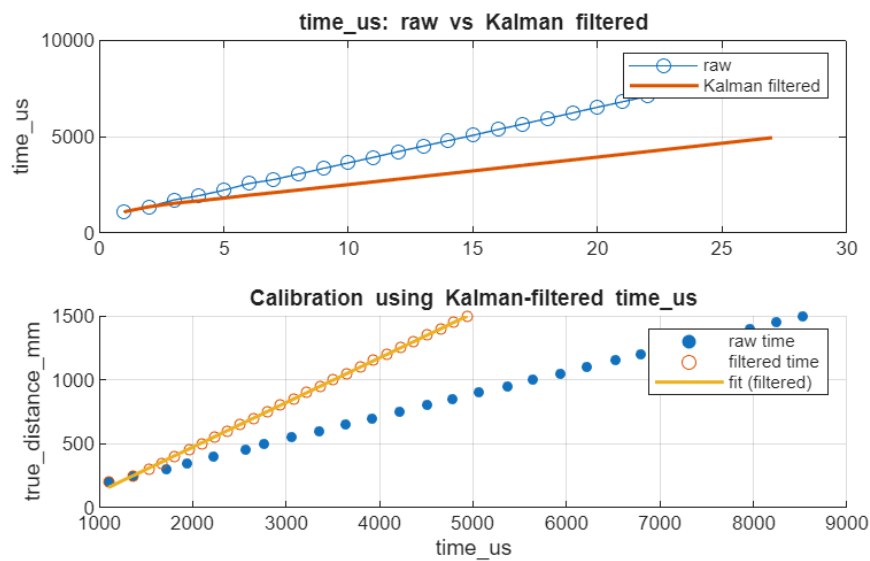
1.1 reduces noise by filtering time_us from one sensor, it works well when the target is stationary but can be sensitive to a single sensor's occasional bad echoes.

1.2 reduces noise by sensor redundancy if one sensor has a noisy or weak echo, the fused estimate is still stable because the second sensor helps anchor the estimate.

Bias / environment sensitivity:

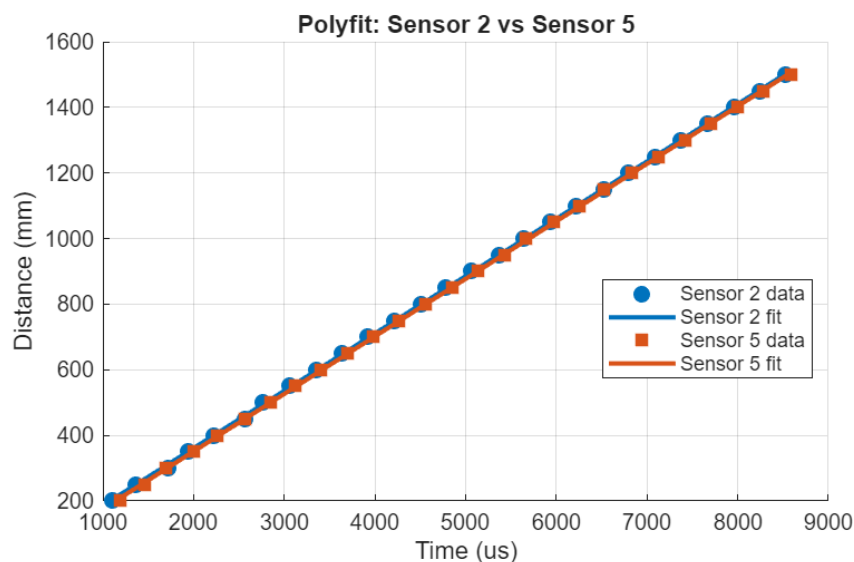
- Ultrasonic time-of-flight depends on the speed of sound, which varies with temperature. Initially the distance was calculated using speed of sound 343 m/s (room temperature). Additionally we had to add piece-wise calibration to get correct distances.
- During calibration, some sensor/target setups showed non-linear behavior at farther distances (notably beyond ~100 cm).
- We also explored quadratic fitting calibration to compensate, but later determined the main issue was weak/unstable echoes when using a small target block. Repeating calibration with a larger target block produced a clean, approximately linear increasing dataset suitable for a linear fit.
- We observed difference in time value when 2 sensors were setup without gap between them vs 1 inch gap between them. The one with some gap worked better.

Project 1.1

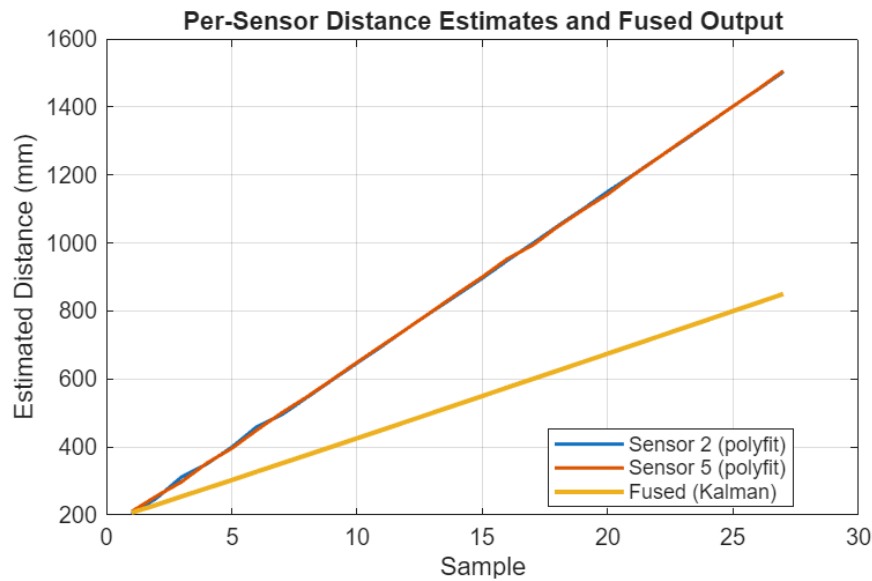


- Top graph (raw vs Kalman-filtered time_us): The Kalman filter smooths the noisy raw echo-time measurements but lags behind when the true time is increasing.
- Bottom graph (Calibration using Kalman-filtered time_us): Using Kalman-filtered time measurements produces a much cleaner, near-linear time–distance relationship and a better linear calibration fit than raw time.

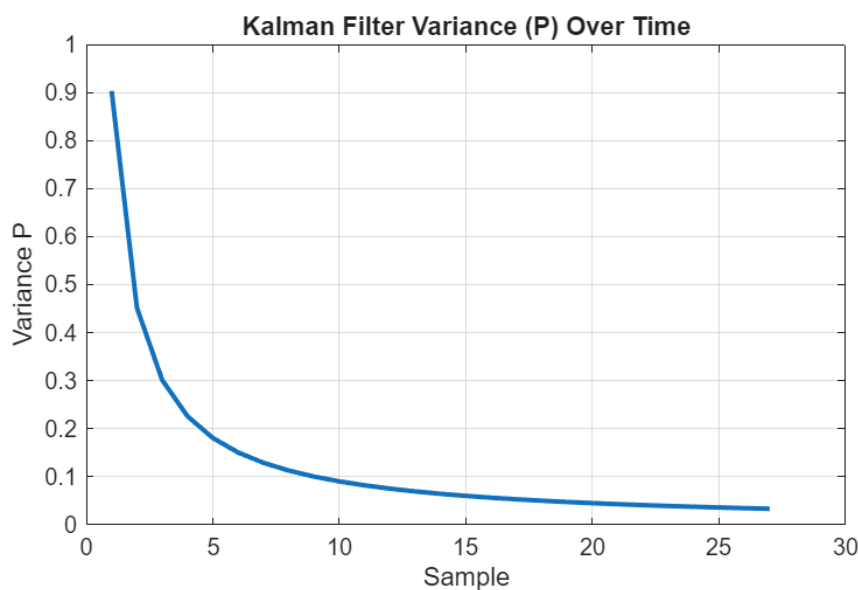
Project 1.2



Both sensors show an almost identical linear time-distance relationship and their fitted lines closely overlap across the full range.



The fused Kalman estimate is smoother but lags below the two per-sensor distance estimates as the distance increases over samples.



The filter uncertainty drops rapidly in the first few samples and then gradually approaches a small steady value, indicating quick convergence. Asymptotically stable.

Future Work:

Adaptive Kalman tuning: dynamically estimate/adjust R based on observed measurement variance or use a non-zero Q when the target may move (reduces lag).

Outlier rejection: before updating the Kalman filter, reject timeouts/spikes using gating (ignore measurements too far from the current estimate).

Better fusion model: instead of fixed R values, assign weights based on signal quality (echo strength/timeout rate) or fuse more than two sensors for extra robustness.

Project 2: Adaptive Cruise Control and Autonomous Lane Keeping

1. Adaptive Cruise Control

1.1 Problem Statement

Requires driving an RC vehicle down a test track as fast as possible without crashes, using an embedded controller based on ultrasonic sensors.

For Adaptive Cruise Control (ACC), the vehicle must:

- Keep the vehicle 30 cm away from obstacles ahead.
- If the obstacle ahead is stationary, stop and hold position 30 cm away from the obstacle.

During the full test sequence: a board is initially placed 30 cm from the start line (the vehicle crosses the line and then reverses to stop at the start line), after the board is removed the vehicle runs down the ramp while maintaining lane centering, finally a board is placed 30 cm from the end line and the vehicle must stop at the end line.

Test Track:



1.2 Technical Approach

- Ultrasonic ranging + calibration: Measure echo pulse width (pulseIn) and convert time $\rightarrow$ distance (mm) using per-sensor linear calibration $d = At + B$, average two front sensors to reduce noise.
- Closed-loop distance regulation (PID): Use a PID controller on the front distance error $e_d = d_{\text{front}} - d_{\text{desired}}$ to generate a throttle command (servo pulse width around 1500 μs).
- Command saturation + safety logic: Constrain throttle to safe bounds (min/max μs), implement a stop band around the desired gap (neutral throttle), and a failsafe mode (constant throttle) if the sensor reading is unreliable/out of range.

1.3 Hardware and Software Implementation

Hardware (Arduino + RC car):

- Steering servo motor on pin 6, ESC (esc) on pin 5.
- Front ultrasonic Sensor 2: trig pin 13, echo pin 12.
- Front ultrasonic Sensor 5: trig pin 11, echo pin 10.

- Right ultrasonic Sensor 3 (used for lane keeping): trig pin 8, echo pin 7.
- Serial logging at 9600 baud.

Software details (ACC path):

- Desired front distance: DESIRED_DIST = 360 mm (300 mm + 60 mm for sensor position)
- Stop window: Stop1 = 330 mm, Stop2 = 410 mm
- Desired side distance: DESIRED_SIDE = 315 mm (300 mm + 15 mm for sensor position)
- Speed limits: MIN_SPEED_US = 1390, MAX_SPEED_US = 1620, neutral = 1500
- Failsafe speed: FAILSAFE_SPEED_US = 1640
- Valid front sensor limit: MAX_VALID_DIST = 700 mm
- Steering PID gains: Kp_steer=4.1, Ki_steer=3, Kd_steer=0.9
- Throttle PID gains: Kp_drive=1, Ki_drive=3, Kd_drive=17
- Steering bounds: constrain(1350, 1650)
- Waits SAMPLE_TIME = 0.05 s each loop.
- Reads Front Sensor 1 (pins 13/12) and Front Sensor 2 (pins 11/10) using pulseIn().
- Converts echo time into distance (mm) using calibration:
 - distance_F1 = CAL2_A * time + CAL2_B
 - distance_F2 = CAL5_A * time + CAL5_B
- Computes front distance: Front_dist = (distance_F1 + distance_F2)/2.
- Validates front reading: sensorValid = (Front_dist < 700 mm).
- If Front_dist is in stop window 330-410 mm, sets VelUS = 1500 and resets PID errors.
- Else (valid + not stopping) → runs throttle PID to track DESIRED_DIST = 360 mm:
 - VelUS = 1500 + PID(Front_dist, 360)
 - Constrains VelUS to 1390-1620 μ s.
- If invalid (Front_dist $\geq$ 700 mm) uses failsafe speed: VelUS = 1640 and resets PID errors.

1.4 Experimental Results

During testing, the serial log reports both front sensor distances and the averaged Front_dist used by ACC. When the vehicle is farther than the desired distance, the PID increases throttle above neutral, when too close, the command decreases toward neutral/reverse (bounded by MIN_SPEED_US).

When Front_dist falls within the stop band around the setpoint, the controller holds neutral throttle (1500 μ s) to reduce oscillation and prevents sudden acceleration by resetting the PID internal error history.

If the front measurement is out of range (Front_dist $\geq$ 700 mm), the controller enters failsafe mode and commands a constant speed (1670 μ s) until valid measurements return.

Exact tracking quality depends on calibration accuracy, ultrasonic target reflectivity/angle, and the chosen PID gains.

Serial log fields (Each Loop):

```
Front 1 = 720.00 | Front 2 = 735.00 | Front = 727.50 | Side =  
310.00 | LaneErr = -5.00 | SteerUS = 1485 | VelUS = 1640  
Front 1 = 492.00 | Front 2 = 508.00 | Front = 500.00 | Side =  
300.00 | LaneErr = -15.00 | SteerUS = 1460 | VelUS = 1605  
Front 1 = 358.00 | Front 2 = 366.00 | Front = 362.00 | Side =  
320.00 | LaneErr = 5.00 | SteerUS = 1520 | VelUS = 1500
```

2. Autonomous Lane Keeping

2.1 Problem Statement

Autonomous Lane Keeping requires the vehicle to stay along the center of a defined lane while driving down the test track. After the start-line reset the vehicle must run down the ramp along the lane center, avoid hitting the boundaries and pass through check points before stopping at the end line. The controller must steer smoothly to reduce lateral error while the vehicle is moving under ACC.

2.2 Technical Approach

- Ultrasonic lateral sensing + calibration: Use a side ultrasonic sensor to estimate lateral distance to the lane boundary and convert time $\rightarrow$ distance with a linear calibration.
- Error-based steering control (PID): Compute lateral error $e_y = d_{\text{side}} - d_{\text{side, desired}}$ and apply a PID steering controller to keep the vehicle centered.
- Actuator mapping + saturation: Map the controller output to steering pulse width (μs) around 1500 (straight) and constrain to prevent over-steering, while running simultaneously with ACC in the same control loop.

2.3 Hardware and Software Implementation

Hardware: The right-side ultrasonic sensor (Sensor 3) is mounted to measure distance to the lane boundary/wall. Steering is actuated through a standard RC servo (pin 6).

Software details (lane-keeping path):

- Reads Right-side sensor (pins 8/7) using `pulseIn()`.
- Converts echo time into distance (mm) using calibration: $\text{dist_Right} = \text{CAL3_A} * \text{time} + \text{CAL3_B}$.
- Computes lane error: $\text{laneError} = \text{dist_Right} - \text{DESIRED_SIDE}$ with $\text{DESIRED_SIDE} = 315$ mm.
- Runs steering PID: $\text{SteerUS} = 1500 + \text{PID}(\text{dist_Right}, 315)$
- Constrains steering command to 1350-1650 μs .
- The computed steering_us is combined with the ACC throttle output and applied via `setVehicle()`.

2.4 Experimental Results

The serial output includes the side distance (Side) and the current lane error (LaneErr). When $\text{Side} > \text{DESIRED_SIDE}$, the controller steers to reduce the gap, when $\text{Side} < \text{DESIRED_SIDE}$, it steers away from the boundary.

Steering saturation prevents over-steering, resulting in smoother lane tracking. Performance depends on side-sensor mounting angle, corridor geometry and the chosen PID gains, high gains can reduce steady-state error but may cause oscillation if the sensor noise is significant.

The vehicle stopped exactly 300 mm before the block.



3. Conclusions and Discussions

3.1 Conclusions

ACC and lane keeping were implemented as two coupled feedback loops running at a fixed sample period.

ACC regulates longitudinal motion by controlling throttle based on averaged front distance from two ultrasonic sensors, while lane keeping regulates lateral position using a right-side ultrasonic sensor and steering PID.

Using two front sensors and averaging improved robustness relative to a single reading and the stop-band logic around the setpoint reduced throttle oscillation near the desired following distance.

3.2 Discussions

We initially attempted to apply a Kalman filter after calibration to smooth the distance measurements, as the filter required several samples to converge, which introduced delay (lag) in the control loop.

At the vehicle's operating speed, this delay reduced responsiveness and caused the controller to react too late to obstacles/lane deviations.

Therefore, we removed the Kalman filter and relied on direct calibrated sensor readings with PID control (plus averaging of the two front sensors) to maintain real-time performance.

We tried several tuning options to reduce the steering overshoot near the 30 cm block by adjusting the PID gains.

Increasing K_p made steering react faster, caused more overshoot, while increasing K_d helped damp the oscillations, reducing K_i was also important because integral windup made the vehicle keep turning even after reaching the desired side distance.

The best results were achieved by lowering K_p and raising K_d enough to stabilize the response.

Future Work:

- Add timestamp-based control (use `millis()` to compute dt) and scale integral/derivative terms explicitly, instead of assuming a fixed sample time.
- Filter ultrasonic measurements (median / moving average / Kalman) and reject outliers/timeouts before feeding the PID.
- Introduce anti-windup for the integral term when throttle/steering saturates.
- Use a more informative lane model (two side sensors or line sensors) to estimate heading relative to lane boundaries.
- Make failsafe behavior more conservative (slow down instead of speeding up) depending on track and safety constraints.

Course Evaluation Confirmation:



My Surveys

Student/Respondent

Nandhini Ramachandran ▾

Thank you for submitting a survey for this course. You should receive an email confirmation but for your records you may want to take a screenshot of the page showing completed before you exit. To continue taking surveys for remaining courses, click the My Surveys button at the top.

AUE-6220-003 : Auto Electronics Integration (Jia)

Automotive Engineering

Starts 10/30/2025 8:00 AM - Ends 12/13/2025 11:59 PM

Completed on 12/12/2025 9:32 PM